GABRIEL POPA

Senior Software Engineer- React • Node (ExpressJS/ NestJS) • AWS
Cluj, Romania | +40 764309991 | popagabriel0523@outlook.com | https://www.linkedin.com/in/gabriel-popa-89ba643a4/

ABOUT ME

Senior Software Engineer with **11+ years of experience** building scalable **React (v16–v18)** frontends and reliable **Node.js (v12–v20)** backend APIs for modern web platforms, turning complex and evolving requirements into stable production releases. Strong expertise in **TypeScript**, **REST/GraphQL APIs**, **PostgreSQL**, **Redis**, **Kafka**, **WebSockets**, and **AWS**, with hands-on experience designing maintainable services and resilient client applications. Known for modernizing legacy systems by migrating **React class components to Hooks**, safely upgrading **Node.js runtimes**, and resolving high-impact issues such as **race conditions**, **memory leaks**, and unstable builds through profiling, contract testing, performance tuning, and controlled rollout strategies.

SKILLS

- **Frontend:** React v16–v18, TypeScript v4–v5, Next.js v12–v14, Redux Toolkit, Zustand, React Query (TanStack Query), React Hooks, Context API, Vite/Webpack, Storybook, Jest, React Testing Library, Cypress, Playwright, HTML5, CSS3, Tailwind CSS, Responsive UI, SPA architecture, component design patterns
- **Backend & APIs:** Node.js v12–v20, Express/NestJS patterns, REST APIs, GraphQL, gRPC basics, WebSockets, event-driven services, JWT/OAuth2 authentication, OpenAPI/Swagger, background workers, job queues, API versioning, middleware architecture
- **Data / Messaging / Search:** PostgreSQL v11–v15, MySQL, Redis (caching/session store), Kafka, RabbitMQ, Elasticsearch/OpenSearch basics, query optimization, indexing strategies, schema design, event streaming, pub/sub messaging
- **Cloud & DevOps:** AWS (EC2, S3, RDS, ECS/EKS basics, IAM basics), Docker, Terraform basics, GitHub Actions/GitLab CI, CI/CD pipelines, Nginx, Linux, containerized deployments, environment configuration, build pipelines
- **Observability / Quality / Security:** OpenTelemetry basics, Prometheus/Grafana basics, Sentry, structured logging, distributed tracing basics, rate limiting, OWASP security practices, Jest, React Testing Library, Supertest, Cypress, Playwright, unit testing, integration testing, contract testing, end-to-end testing, and error monitoring

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania *(Jan2024 – Dec 2025)*

- **Led** development of a multi-tenant dashboard using **React**, **TypeScript**, and **Next.js**, refactoring unstable screens into reusable components that improved UI consistency and reduced tenant-specific issues.
- **Improved frontend performance** by profiling slow-rendering components, memoizing expensive selectors, and adding **React Query** caching, which reduced UI freezes by **45%** and kept large tables and filters responsive.
- **Applied** modern **React** concurrent patterns such as **useTransition** to prevent non-urgent updates from blocking the interface, resulting in smoother user interactions during heavy data processing.
- **Enhanced** real-time user experiences by improving **WebSocket** reliability with reconnection handling and buffered event queues, keeping live status updates stable during temporary network disruptions.
- **Built** scalable backend services with **Node.js**, **NestJS**, and **TypeScript**, delivering maintainable **REST** and **GraphQL** APIs with clear service boundaries and strong modular design.
- **Modernized** a legacy **Node.js** API from v14 to v20 through staged upgrades, compatibility validation, and safe deployment planning, avoiding disruption to critical payment workflows.
- **Resolved** a production memory leak by identifying issues in in-memory caching and stale event listeners through heap analysis and load testing, restoring service stability under sustained traffic.
- **Prevented** duplicate transaction processing by implementing idempotency controls, strengthening **PostgreSQL** constraints, and refining retry handling for downstream service failures.
- **Strengthened** application security by implementing **JWT** and **OAuth2** authentication, **RBAC** authorization, audit logging, and request rate limiting to improve access control and platform protection.

- **Integrated Kafka** and **RabbitMQ** to support asynchronous workflows for billing events, notifications, and background processing, improving reliability and reducing coupling across services.
- **Introduced OpenAPI** contract testing into **GitHub Actions** CI/CD pipelines, enabling frontend and backend teams to release independently with lower integration risk.
- **Improved** production observability through structured logging, **Sentry**, and **OpenTelemetry** metrics sent to **Prometheus/Grafana**, making incidents easier to detect, trace, and resolve.
- **Containerized** backend services with **Docker** and deployed them on **AWS EC2/ECS**, automating build and release workflows through CI pipelines to improve deployment consistency.
- **Implemented Elasticsearch/OpenSearch**-based search capabilities, enabling fast tenant-scoped queries and scalable filtering across large operational datasets.

Software Engineer

Next Apps | Ghent, Belgium *(Nov2020 – Nov 2023)*

- **Built** customer-facing workflows with **React** and **TypeScript**, improving form reliability, validation consistency, and accessibility across complex product journeys.
- **Partnered** with design teams to create a reusable **React** component library, reducing duplicated UI work while supporting product-specific customization needs.
- **Implemented** the new **JSX Transform**, removing unnecessary **React** imports and simplifying component code while slightly improving bundle size and build efficiency.
- **Migrated** shared frontend state management from **Redux** to **Zustand**, reducing state-management boilerplate by **35%** and making store logic easier to maintain with **React Hooks**.
- **Improved** frontend build tooling by simplifying heavy **Webpack** configuration and optimizing build steps, which sped up local development and reduced CI build instability.
- **Modernized** a legacy **Express.js** API by migrating it to **NestJS**, reorganizing routes into modular controllers and services to improve maintainability and code structure.
- **Developed Node.js** services with versioned **REST** endpoints and deprecation strategies, supporting backward compatibility while reducing coupling between API consumers and backend changes.
- **Implemented Redis** caching for high-traffic endpoints using safe invalidation rules and TTL strategies, reducing database load while keeping data accurate.
- **Resolved** intermittent **502/504** errors by tuning **Nginx** timeout settings, optimizing slow database queries, and adding latency monitoring around critical user workflows.
- **Fixed** a data consistency issue caused by out-of-order event processing by enforcing ordering keys and building a background reconciliation process.
- **Improved** asynchronous job reliability by adding retry policies, dead-letter handling, and monitoring dashboards, making failed or stuck workloads easier to detect and recover.
- **Strengthened** authentication flows by tightening token validation, adding rate limiting for abuse protection, and safely rotating secrets across services.
- **Introduced** structured logging and correlation IDs across frontend and backend systems, making it faster to trace production issues from user actions to backend handlers.
- **Improved** release reliability by enforcing code review checklists, maintaining test coverage thresholds, and documenting rollback steps for higher-risk production changes.

Software Developer

Datamart | Stockholm, Sweden *(Feb 2016 – Sept 2020)*

- **Built** internal analytics dashboards using **React** and **Node.js**, integrating large datasets into responsive interfaces with server-side pagination and caching to improve usability and performance.
- **Supported** the refactoring of a large **Node.js** API into smaller modular services behind an API gateway, improving maintainability while preserving stability for existing integrations.
- **Improved reporting performance** by investigating slow pages, adding **PostgreSQL** indexes, rewriting heavy joins, and reviewing execution plans, reducing query latency by **41%**.
- **Modernized** legacy frontend code by converting **React Class Components** to **Function Components** after the move to hooks-based patterns, simplifying lifecycle logic and improving maintainability.
- **Resolved** a difficult data issue by identifying timezone parsing differences between the UI, API, and database, then standardizing date handling to restore reporting accuracy.

- **Implemented** background job processing for large file exports, replacing synchronous requests that frequently timed out and adding status polling to improve user visibility.
- **Added** automated integration tests around core business rules, reducing regressions and helping maintain consistent behavior during frequent releases.
- **Improved** CI pipeline reliability by separating fast unit tests from slower integration tests, reducing flaky build failures and speeding up feedback cycles.
- **Introduced** audit logging for data update workflows, enabling teams to trace user actions and review change history during issue investigations.
- **Reduced** frontend bundle size by applying **Webpack** code splitting and removing unused dependencies, improving page load times and overall client-side performance.
- **Assisted** with production releases by introducing smoke tests and staged deployments, helping identify configuration issues before they affected users.

Junior Software Developer

Berg Software | Timișoara, Romania *(Sep2014 – Mar 2016)*

- **Built** early SPA modules using **Node.js** and **Express**, creating **REST** endpoints with consistent response handling and basic error management across backend workflows.
- **Improved** legacy web pages by adding modular **JavaScript**, client-side validation, and clearer form feedback, which reduced user input errors and support requests by **28%**.
- **Helped** resolve a recurring logout issue by fixing session cookie settings and aligning server-side session timeouts with browser behavior, improving login reliability for users.
- **Developed** reusable UI components and shared helper utilities, reducing duplicated code and making common frontend updates faster and easier for the team.
- **Improved** page load performance by enabling asset compression, configuring browser caching, and removing blocking scripts that delayed initial rendering.
- **Supported** deployment automation by writing simple release scripts and maintaining deployment documentation, helping the team deliver updates more safely and consistently.
- **Assisted** with production issue resolution by reviewing logs, reproducing bugs locally, and applying targeted fixes under guidance from senior engineers.
- **Improved** database performance by writing safer **SQL** queries and adding basic indexes to support admin dashboards as data volume increased.
- **Used** a structured debugging approach by documenting reproduction steps, reviewing logs, and delivering small, low-risk fixes instead of large refactors.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (*2010 – 2014)*

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERITIFICATIONS

AWS Certified Developer – Associate — 2024

- Demonstrated practical capability in building, deploying, and maintaining **Node.js-based backend services** on **AWS**, with focus on service integration, monitoring, secure configuration, and production-ready cloud operations.

OpenJS Node.js Application Developer (JSNAD) — 2023

- Validated hands-on understanding of **Node.js** application development, including asynchronous programming, package management, debugging, testing, and building maintainable backend services with modern JavaScript patterns.

PostgreSQL for Developers — 2022

- Demonstrated working knowledge of relational modeling, query optimization, indexing strategies, transaction-aware data access, and performance-focused backend persistence patterns commonly used in **API-driven Node.js platforms**.